VC

AI-Based Face Recognition for VC Cabin Entry

Security-Hardened Visitor Identification Appliance

AI Prototype • MCA Project

Rajat Sharma • MCA-FS • RU-26-01267

Software-only prototype — detects, recognizes, logs and notifies.

No door or lock is controlled; the human makes the access decision.

1. Problem Statement & Objective

A VC (Vice-Chancellor) cabin entry currently has no reliable record of who entered and when. Manual register entries are slow, inaccurate and easy to forge. The objective is to build a system that identifies students and faculty at the entry point automatically — and that behaves like a real security appliance rather than a demo: decisions rest on accumulated evidence, attacks are detected, failures are visible, and every action is auditable.

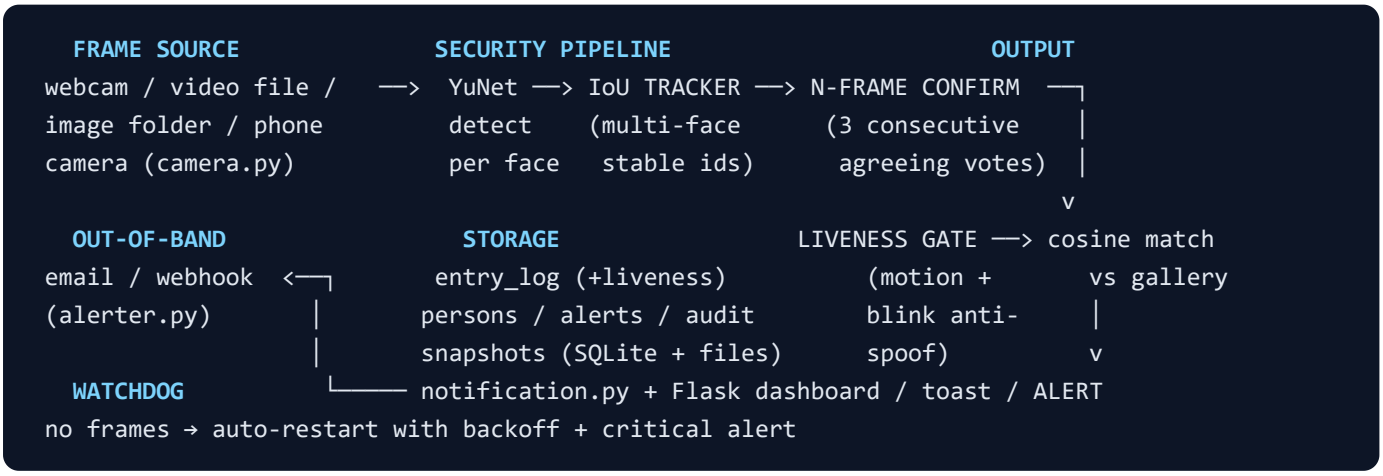
> **Scope boundary (important):** this prototype is **software only**. It detects a face, decides who it is, records it, and raises an alert. It **never opens a door or controls any lock** — the security guard / staff member reviews the alert and physically grants or denies access.

Objectives

- Detect and **track** faces reliably at the entry point — multiple visitors simultaneously, stable identities across frames
- Recognize enrolled students and faculty on **sustained evidence** (N-frame confirmation), not a single lucky frame
- Reject presentation attacks — printed photos and frozen screens must never be accepted as a live person
- Log every entry with identity, department, ID, time, result and liveness verdict
- Notify staff instantly — on screen, and out-of-band by email or webhook when configured
- Stay available: detect camera failures and recover automatically
- Keep a complete, tamper-evident audit trail; keep data local and short-lived
- Work with or without a physical webcam (video, images, or phone camera)

2. System Architecture

The system runs entirely on one machine. A background thread pulls frames through a six-stage recognition pipeline, a Flask server (Waitress in production) serves the login-gated dashboard, and everything is stored in SQLite.



Module	Responsibility
`camera.py`	Frame sources: webcam index, video file, image folder, or a live network stream (phone camera). Includes a low-latency MJPEG reader.
`face_detection.py`	YuNet face detection wrapper (ONNX).
`tracker.py`	Greedy IoU multi-face tracker. Gives each visitor a stable track id and accumulates per-track evidence (identity votes, liveness history).
`liveness.py`	Passive anti-spoofing: landmark-motion + eye-region blink energy → `live / unknown / spoof` verdict.
`face_recognition.py`	SFace 128-D embedding + cosine best-match against the gallery; threshold tunable (`VC_MIN_COSINE` , default 0.363).
`pipeline.py`	Background thread: read → detect → track → confirm → liveness gate → match → log → notify; publishes the per-track workflow snapshot.
`alerter.py`	Out-of-band alert channels: SMTP email + generic/Discord webhook, per-channel rate limiting, critical-event bypass.
`database.py`	SQLite: persons (with consent), entry log (with liveness verdicts), alerts with acknowledge/reject decisions, append-only audit log.
`backup.py`	Consistent SQLite-backup-API archives (database + snapshots + manifest) with pruning and integrity verify.
`enrollment.py`	Consent-gated registration; multiple photos averaged into one embedding.
`notification.py`	Windows toast notification (fire-and-forget, can be disabled).
`app.py`	Flask dashboard behind a login gate, enrollment API, image-upload analysis, live MJPEG feed, security headers, camera watchdog.
`serve.py`	Production entrypoint (Waitress WSGI server).
`evaluate.py`	Confusion-matrix evaluation → `reports/evaluation.csv` .
`threshold_sweep.py`	Threshold sweep over genuine/imposter pairs → FAR/FRR report with a proven-safe interval → `reports/threshold_sweep.{csv,md}` .

3. Technology Stack

Layer	Choice	Why
Face detection	OpenCV **YuNet** (ONNX, 2023-Mar)	Accurate on small/angled faces, fast on CPU
Face recognition	OpenCV **SFace** (ONNX, 2021-Dec)	Compact 128-D embeddings, vendor-published threshold

Similarity	Cosine similarity, threshold tunable (default **0.363**)	Vendor operating point, later re-evidenced by our own sweep
Tracking	Greedy IoU tracker (custom, `tracker.py`)	Stable per-visitor identity across frames without heavy dependencies
Liveness	Passive PAD: landmark motion + blink energy (custom)	No extra model or IR camera needed on plain RGB
Backend	**Flask** + **Waitress** (production)	Simple, local, no external services
Storage	**SQLite**	Zero-setup, file-based, fully local
Frontend	Server-rendered HTML + vanilla JS	No build step, no framework, live polling
Models	OpenCV Model Zoo (ONNX)	Openly licensed, runs offline via OpenCV DNN
Testing	**pytest** (45 tests) + **GitHub Actions** CI	Regression safety on every push, Python 3.11 & 3.12
Deployment	**Docker** image + compose, health-checked	Repeatable install on a college server
Language	Python 3	De-facto standard for OpenCV

> **No cloud, no internet required at runtime.** Inference runs locally through OpenCV DNN. All personal data stays on the machine.

4. How the Recognition Pipeline Works

1. **Capture a frame.** From the configured source (webcam, video, image folder, or phone camera over Wi-Fi).
2. **Detect all faces (YuNet).** Every face in the frame is processed independently — the system handles multiple visitors at once. Frames with no face are skipped silently.
3. **Track across frames (IoU tracker).** Each face gets a stable track id; detections are attached to existing tracks by intersection-over-union, so one visitor is one track, not a stream of unrelated frames.
4. **Embed the face (SFace).** The face is *aligned* using its 5 landmarks, then encoded into an L2-normalized 128-D vector and matched against the gallery by cosine similarity.
5. **N-frame confirmation.** A MATCHED or UNKNOWN verdict requires **3 consecutive frames agreeing** (`VC_CONFIRM_FRAMES`). One blurred or lucky frame can no longer create or deny a log entry.
6. **Liveness gate.** Landmark motion and blink energy are accumulated per track. On live sources, a confirmed match that is not verified live is **held** (up to 20 frames) and then logged unverified; a photo/screen attack is logged as a presentation attack and raises a **critical alert**.
7. **Log the entry.** Name, category, department, ID, timestamp, confidence, result **and liveness verdict** are written to `entry_log` with a cropped snapshot.
8. **Notify staff.** Desktop toast + dashboard alert, and email/webhook when configured. Staff then **Acknowledge** or **Reject**.

Live workflow visibility

The dashboard shows the pipeline decision process in real time. Each tracked face is rendered as a stage strip — **DETECT** → **TRACK** → **CONFIRM** → **LIVENESS** → **MATCH** → **LOG** — green when a stage passed, cyan while evidence is accruing, amber while holding, red-blinking on a spoof verdict. This makes the ~1 second confirmation window visible instead of looking like a malfunction, and shows the operator exactly why an entry has (or has not) been logged.

Duplicate suppression

A 15-second cooldown (`VC_COOLDOWN`) prevents the same person (or the same unknown visitor) from being logged repeatedly while they stand in front of the camera. The cooldown is keyed per identity, so two different visitors are never throttled against each other.

When a face is not recognized

The entry is logged as `UNKNOWN` with the raw cosine score — **never force-matched** to the nearest enrolled person. Unknowns get the same N-frame treatment as matches, so detection flicker cannot spam the alert list.

5. Recognition Security: Evidence, Not One-Shot Decisions

Threshold sweep — an evidenced operating point

The SFace vendor default (0.363) is a reasonable start, but it is not a measurement on *this* camera or *this* gallery. `threshold_sweep.py` measures score distributions — **genuine pairs** (same identity across augmented cross-condition variants: flip, rotation, brightness, blur, scale) versus **imposter pairs** (every cross-identity combination) — and sweeps the threshold from 0.20 to 0.70, reporting FAR, FRR, precision, recall and F1 at each point.

56	64	0.21–0.69	0.450
GENUINE PAIRS	IMPOSTER PAIRS	PROVEN-SAFE INTERVAL	RECOMMENDED POINT

Operating point	Threshold	FAR	FRR	Recall	F1
Security (FAR ≤ 1%)	**0.450**	0.0000	0.0000	1.000	1.000
Balanced (best F1)	**0.450**	0.0000	0.0000	1.000	1.000
Vendor default (SFace)	0.363	0.0000	0.0000	1.000	1.000

Every threshold in **[0.21, 0.69]** achieved zero false accepts at full recall on the measured pairs; the recommendation is the interval midpoint (largest margin from both the nearest imposter and the nearest genuine score). The vendor default 0.363 sits *inside* the proven-safe interval, so it was kept as the shipped default — now as an evidenced choice, not an assumption. The sweep is operator-tunable (`VC_MIN_COSINE`) and re-runnable on-site with real probe photos.

> **Honest limitation:** augmented variants are a surrogate for a real second capture session, and 2 identities cannot bound a production FAR. The generated report says so, and instructs re-running the sweep with probes from the actual installation camera before freezing the threshold.

Passive liveness (anti-spoofing)

Per track, two cheap signals are computed from the ordinary RGB feed:

- **Landmark motion** — frame-to-frame displacement of the 5 YuNet landmarks, normalized by face size (scale- and distance-invariant);
- **Blink detection** — Laplacian texture energy of the two eye crops relative to the whole face (lighting-invariant). A blink drops the energy sharply below the personal baseline and back.

A printed photo or frozen screen shows neither motion nor blinks and is judged a **spoof** — but only after **≥10 frames of evidence**, so a genuinely still visitor is never branded an attacker on 3 frames. In `enforced` mode (the default on live sources) a spoof verdict **never becomes MATCHED**, even for an

enrolled person; it is logged as a presentation attack with a critical alert. Modes: `enforced` | `monitor` (label only) | `off`.

> **Honest limitation:** passive single-camera checks cannot distinguish a high-quality **video replay** of a blinking person from the real person. The threat model (`docs/SECURITY.md`) documents this and recommends a discretionary guard challenge (ask the visitor to turn their head) for higher-risk installations — which the live workflow view and camera feed make easy to supervise.

6. Dashboard Features

Access control

Every page and API (except `/login`, `/health` and static assets) sits behind a single admin password (`VC_ADMIN_PASSWORD`; generated randomly and printed to the log if unset — the dashboard is never left open). Five failed attempts trigger a 60-second per-IP lockout; sessions are HttpOnly/SameSite cookies with a 12-hour lifetime; cross-site mutating requests are refused (CSRF defence).

Screen	What it shows
Dashboard	Four stat cards, the **live camera preview** with pipeline counters (source, liveness mode, camera age, armed alert channels), the **live recognition-workflow panel**, a camera-outage banner when degraded, the upload-analyze panel, pending notifications, and the recent visitors table with **liveness chips**.
Visitor Logs	Full entry history — photo, visitor, category, department, ID, time, result, confidence **and liveness verdict** (LIVE / UNVERIFIED / SPOOF?).
Students & Staff	All enrolled identities with status, and a deactivate action.
Notifications	Pending visitor alerts with **Acknowledge** / **Reject** buttons, spoof events flagged critical, plus a "recently resolved" decision trail.
Audit Trail	Append-only record of every operator action: logins (including failures), acknowledgements/rejections, enrollments, deactivations, recognition toggles, retention purges, camera outages and restarts.
Register Person	Consent-gated enrollment: name, category, department, ID and one or more photos. Multiple photos are averaged into a single embedding for accuracy.

Two ways to use it

- **Live monitoring** — point a webcam (or a phone camera acting as one) at the entry; the dashboard shows the feed, the workflow strip, and logs people as they pass.
- **Upload & analyze** — drop in a still image and get an instant identity verdict. Useful when no camera is available.

Notification handling

Each alert can be **Acknowledged** (handled/allowed) or **Rejected** (denied). Both actions clear the notification and record the decision for audit. If nobody acts on an alert, it **clears itself after 24 hours**, so the notification list never grows without bound.

7. Operations: Alerts, Watchdog, Backups, Hardening

Out-of-band alert routing

On-screen alerts only help someone who is watching. `alerter.py` routes events to an **SMTP mailbox** and/or a **generic webhook** (Discord/Slack/IFTTT/Zapier-compatible), configured purely by environment variables. Delivery is **rate-limited per channel** (`VC_ALERT_MIN_INTERVAL` , default 60 s) so a noisy stream cannot flood the guard, while **critical events always deliver** — suspected spoofing and camera outages. A failing channel (bad credentials, network down) is logged and never crashes the pipeline. With no channels configured, behavior is exactly as before.

Camera watchdog

A security camera that silently stops is worse than none. The watchdog watches frame flow on live sources: **no frames for 25 seconds** (`VC_STALL_SECONDS`) triggers an automatic pipeline restart with backoff, a **red "camera outage — entry unmonitored" banner** on the dashboard, `camera.outage` / `camera.recovered` audit entries, and a critical alert. The degraded state is visible rather than silent.

Backups

`python backup.py` produces a timestamped, compressed archive of the database (via SQLite's backup API — consistent even while the app is writing) plus all snapshot images and a manifest with row counts and restore steps. Old archives are pruned (`VC_BACKUP_KEEP` , default 30); `--verify` integrity-checks the live database. The README ships Task Scheduler (Windows) and cron (Linux) recipes for daily backups.

Transport & hardening

- Security headers on every response: `X-Content-Type-Options: nosniff` , `X-Frame-Options` , `Referrer-Policy` ; API responses are `no-store` .
- HTTPS by reverse proxy: the production server (Waitress) deliberately does not terminate TLS itself — ready-to-use **Caddy** (automatic certificates) and **nginx** configurations are documented, with the app bound to localhost behind the proxy.
- Retention: entries, alerts and snapshots purged after 30 days, audit rows after 90 — automatic, on startup and every 6 hours.
- **45 automated tests** (pytest, no camera needed) + GitHub Actions CI on every push (Python 3.11 & 3.12); Docker image with health check.

8. Live Camera — Phone as a Webcam

The built-in webcam on this machine was not available, so the system was built to accept a phone camera as a live source. Running an app such as **IP Webcam** or **DroidCam** on the phone exposes the camera as an MJPEG video stream over Wi-Fi, which the system consumes directly.

```
py app.py --source http://<phone-ip>:8080/video
```

> **Engineering note:** OpenCV's FFMPEG backend buffers HTTP streams heavily — a phone camera serving ~15 fps was read at under **0.25 fps** through `VideoCapture`. A dedicated MJPEG multipart reader (`camera.MjpegStream`) was written to parse the stream directly, raising throughput to ~**4.3 fps** at 640×480. The pipeline also auto-reconnects if the phone drops off the network, and the watchdog restarts it if frames stop altogether.

9. Performance & Security Evaluation

Self-test (re-identification)

Metric	Value	Metric	Value
Samples	2	True Positives (TP)	2
False Positives (FP)	0	False Negatives (FN)	0
Accuracy	100%	Precision	100%
Recall	100%	F1 Score	100%

Live video evaluation (20 probe frames from the demo clip)

10
CORRECTLY RECOGNIZED

10
CORRECTLY NOT MATCHED

12
FACES DETECTED IN TOTAL

Scenario	Outcome
Enrolled person in frame	Recognized — cosine 0.37 to 0.67, above the 0.363 threshold
Un-enrolled background face	Rejected — cosine near zero, logged as UNKNOWN for manual review
Frames with no face (wall/ceiling)	Ignored silently — no log entry, no false alarm

Security-layer verification (live)

Test	Method	Result
N-frame confirmation	45 automated tests + live run	Verdicts need 3 consecutive agreeing frames; streak resets on disagreement
Photo/screen attack	Static photos fed as a live image source	Flagged **SPOOF?** in the workflow panel, logged unverified, never MATCHED
Camera outage	Source stopped mid-run	Outage banner + critical alert + auto-restart + audit entries
Operator audit	Login / ack / reject / enroll / toggle	Every action in the append-only audit trail
Backup integrity	`backup.py` + `--verify`	Consistent archive with manifest; PRAGMA integrity check OK

> **Observed limitation:** matching quality depends on how large the face is in the frame. During phone testing at 640×480 the face was only ~55–90 px wide, which pushed the cosine score below threshold. Holding the camera closer, or using a higher stream resolution (1280×720 or above), restores accurate recognition — and the N-frame confirmation now absorbs single-frame dips automatically.

> **Design strength:** the system errs on the side of caution at every stage. A low-confidence face is logged as UNKNOWN and flagged for a human rather than force-matched; a confirmed identity without liveness evidence is held, not accepted; in the evaluation above, **zero false acceptances** occurred.

10. Security Model, Privacy & Ethical Considerations

`docs/SECURITY.md` contains the full threat model (T1–T10 with mitigations and residual risks); `docs/PRIVACY_NOTICE.md` is a plain-language, DPDP-aligned data-practices notice. The essentials:

Threat model summary

Threat	Mitigation	Residual risk (honest)
Wrong identity from one blurred frame	Tracking + N-frame confirmation	Persistent similar-looking impostor — bounded by threshold + liveness
Printed photo attack	Motion + blink PAD; blocked from MATCHED in enforced mode	Very high-res photo with hand jitter may read "unknown", never "live"
Video replay of a blinking person	Passive PAD sees genuine motion/blinks	**Can pass** — documented; guard challenge recommended
Dashboard credential theft	Lockout, session hardening, CSRF defence, audit	Single-factor by design (operator appliance)

Camera sabotage / outage	Watchdog: alert + banner + auto-restart	Stall window is unmonitored — made visible, not silent
Evidence loss / tampering	Audit trail, retention, verified backups	Host-level attacker can alter the DB — keep backups off-host
Network eavesdropping	Reverse-proxy TLS, security headers	Built-in server is plain HTTP — do not expose directly
False accepts from a bad threshold	On-site threshold sweep, FAR ≤ 1% target	Sample statistics are small — re-measure on site

Consent-first enrollment

Enrollment requires an explicit consent checkbox. No face is enrolled without it. Face embeddings are stored as binary blobs, not photographs.

Data locality, retention and rights

- All data lives locally: `database/vc_entry.db` plus snapshot images in `static/snapshots/`; **nothing leaves the machine** except operator-configured alert summaries (text only, never embeddings)
- Automatic retention: entries/alerts/snapshots 30 days, audit 90 days
- Data-subject rights: deactivation/deletion removes the person's embedding, photo and profile; full erasure = delete DB + snapshots + backups
- Snapshots are capped (240 most recent) and pruned automatically

Human-in-the-loop by design

The software produces a *recommendation*, never an action. Every alert requires a human Acknowledge/Reject, and no physical access control is driven by the system.

Limitations and bias

- Accuracy depends on lighting, distance and face size in frame
- Trained primarily on adult faces — performance on very young or elderly subjects may differ
- The enrolled gallery determines who can ever be recognized; unknown faces are never guessed
- Liveness is *passive*: it stops photo/screen attacks but not a high-quality video replay (see threat model)

What this prototype does NOT do

No door control, no lock control, no automated entry grant. The prototype stops at identification, logging and notification.

11. How to Run

```
python -m pip install -r requirements.txt
py app.py                            # webcam / sample data
py app.py --source visitor.mp4       # a video file
py app.py --source http://10.0.17.58:8080/video   # phone camera
py serve.py                          # production (Waitress)
```

Then open `http://127.0.0.1:5000` and log in with the admin password (from `VC_ADMIN_PASSWORD` or the log file).

Command	Purpose
`py app.py`	Start with the default source
`--source`	Webcam index, video file, image folder, or stream URL
`--port` / `--no-toast`	Dashboard port / disable Windows notifications
`py serve.py`	Production WSGI server (behind a TLS reverse proxy)
`py backup.py [--keep N] [--verify]`	Back up / integrity-check the database
`py threshold_sweep.py [--dir D]`	Re-measure the match threshold on-site
`py evaluate.py --self-test`	Re-run the accuracy evaluation
`py enrollment.py --image ... --name ...`	Enroll from the command line
`docker compose up -d`	Run containerized with named volumes

Key configuration

Variable	Default	Meaning
`VC_ADMIN_PASSWORD`	generated	Dashboard login (random printed to log if unset)
`VC_LIVENESS`	`enforced`	Anti-spoof: enforced / monitor / off
`VC_CONFIRM_FRAMES`	3	Consecutive agreeing frames for a verdict
`VC_MIN_COSINE`	0.363	Match threshold (tune from the sweep report)
`VC_STALL_SECONDS`	25	No frames → watchdog restart + alert
`VC_ALERT_EMAIL_TO` / `VC_ALERT_WEBHOOK_URL`	unset	Out-of-band alert channels

`VC_COOLDOWN`	15 (seconds)	Duplicate-entry suppression window
`VC_RETENTION_DAYS` / `VC_AUDIT_RETENTION_DAYS`	30 / 90	Automatic data purge windows
`VC_SOURCE` / `VC_MAX_WIDTH`	auto / 960 px	Frame source / downscale width

The README documents the full environment-variable reference (24 variables), alert-channel setup, HTTPS proxy configurations, and backup scheduling.

12. Limitations & Future Work

Current limitation	Planned improvement
Passive liveness cannot stop a video replay of a blinking person	Active challenge-response (automated head-turn prompts) or an IR/depth camera for certified PAD
Sample-data threshold statistics are small (2 identities)	On-site sweep with real probes; per-person adaptive thresholds
Single machine, single camera	Multi-camera support across cabin entry points, with role-based operator logins
Single-factor admin login	SSO / 2FA via the reverse proxy; per-operator credentials
Manual enrollment only	Bulk import from the college student database / ID cards
Local SQLite only	Optional central server with an audit dashboard for security staff
Watchdog restarts re-open the same device	Health metrics dashboard + SNMP/email diagnostics for hardware failures

Earlier limitations — single-face-per-frame decisions, flat single-frame thresholds, and photo-only enrollment with no liveness — have been resolved in this revision by the tracker, N-frame confirmation, threshold sweep and passive anti-spoofing described above.

13. Project Deliverables

#	Deliverable	Status
1	Face detection pipeline (YuNet)	Complete
2	Face recognition & matching (SFace + cosine)	Complete
3	Consent-gated enrollment	Complete
4	Visitor entry logging (SQLite) with liveness verdicts	Complete

5	Login-gated dashboard: camera, workflow view, logs, stats, alerts, audit trail	Complete
6	Acknowledge / Reject with 24-hour auto-expiry	Complete
7	Windows toast notifications	Complete
8	Phone-as-webcam live streaming (low-latency MJPEG)	Complete
9	Multi-face tracking + N-frame confirmation	Complete
10	Passive liveness / anti-spoofing with presentation-attack alerts	Complete
11	Threshold sweep tool + FAR/FRR evidence report	Complete
12	Email / webhook alert routing with rate limiting	Complete
13	Camera watchdog: outage detection, auto-restart, critical alerts	Complete
14	Backup utility (consistent archives, prune, verify)	Complete
15	Append-only operator audit trail	Complete
16	Accuracy evaluation & metrics report	Complete
17	Threat model + privacy notice documentation	Complete
18	45-test pytest suite + GitHub Actions CI + Docker deployment	Complete
19	Source code on GitHub	Complete

Repository

```
https://github.com/Argyrys/Face-Recognition-System
```

> **Conclusion:** the system reliably detects, tracks and identifies visitors on sustained evidence rather than single frames, rejects photo and screen presentation attacks, stays visible and self-recovering when the camera fails, routes alerts beyond the screen, preserves a tamper-evident audit trail with verified backups — while keeping every access decision firmly in human hands and every claim about its own limits honest.